

Review of Common Pandas Functions with Practical Examples

Pandas is a core Python library for structured data manipulation and analysis, providing a comprehensive set of functions to handle data cleaning, transformation, and exploration. Below is a structured review of essential pandas functions—including those for missing value management, grouping, indexing, data type conversion—and extended commonly used operations.

1. Missing Value Analysis & Handling

Missing values are ubiquitous in real-world datasets, and pandas offers intuitive tools to identify and resolve them:

- **Calculating Missing Value Percentage per Column**

To compute the percentage of missing values for each column, combine `isnull()` (detects NaN/None values), `sum()` (counts missing values per column), and `mean()` (calculates the proportion of missing values relative to total rows). Format the output to display percentages with two decimal places:

```
Code block
1  import pandas as pd
2
3  # Sample DataFrame for demonstration
4  df = pd.DataFrame({
5      'ID': [1, 2, None, 4, 5],
6      'Value1': [10.2, None, 15.5, None, 20.1],
7      'Value2': [0, 1, 1, 0, None]
8  })
9
10 # Compute missing value percentage for each column
11 missing_pct = (df.isnull().sum() / df.shape[0]) * 100
12 # Print formatted results
13 for col in missing_pct.index:
14     print(f'{col}: {missing_pct[col]:.2f}% missing')
```

- **Filling Missing Values (`fillna()`)**

`fillna()` is a versatile function to replace missing values with scalars, aggregated values, or column-specific values:

- **Scalar fill:** Replace missing values in a numerical column with a fixed value (e.g., 0 for duration-related metrics):

```
Code block
1  # Fill missing values in 'Duration' column with 0
2  df['Duration'] = df['Duration'].fillna(0)
```

- **Group-wise fill:** Fill missing values using group-level statistics (e.g., median/mean of a categorical group). For example, fill missing 'Sales' values with the median 'Sales' of their 'Category' group:

Code block

```
1 # Fill missing Sales with category-specific median
2 df['Sales'] = df.groupby('Category')
  ['Sales'].fillna(df.groupby('Category')['Sales'].median())
```

- **Column-specific fill (dictionary input):** Fill missing values in different columns with distinct aggregates (e.g., 'Revenue' with its column mean):

Code block

```
1 # Fill missing Revenue values with the column's mean
2 df = df.fillna({'Revenue': df['Revenue'].mean()})
```

2. Indexing & Selection

Pandas provides two primary methods to select rows/columns, critical for accessing specific data points:

- **iloc (Integer-based Indexing):** Selects data by 0-based integer position. Use `iloc[n]` to retrieve the nth row (e.g., `iloc[1]` returns the second row):

Code block

```
1 # Access the 2nd row (integer index 1) after group-wise filling
2 selected_row = df.iloc[1]
3 print(selected_row)
```

- **loc (Label-based Indexing):** Selects data by row/column labels (not just integer positions). Use `loc[label]` to retrieve the row with the specified label (e.g., `loc[2]` returns the row labeled 2):

Code block

```
1 # Access the row labeled 2 after filling Revenue values
```

```
2 labeled_row = df.loc[2]
3 print(labeled_row)
```

3. Data Type Conversion (astype())

After cleaning data, convert column data types to align with analytical requirements (e.g., from `float64` to `int64` for integer-valued metrics like minutes):

Code block

```
1 # Convert 'Duration' column from float64 to int64
2 df['Duration'] = df['Duration'].astype('int64')
3 # Verify the converted data type
4 print(f"Data type of 'Duration': {df['Duration'].dtype}")
```

4. Extended Essential Pandas Functions

Beyond the above, these functions are integral to comprehensive data exploration and manipulation:

- **groupby() (Grouped Aggregation):** Aggregate data by categorical columns (e.g., sum, count, or average values per group):

Code block

```
1 # Calculate total Sales per Category
2 category_sales = df.groupby('Category')['Sales'].sum()
```

- **dropna()**: Remove rows/columns containing missing values (use `axis=0` for rows, `axis=1` for columns):

Code block

```
1 # Drop rows with any missing values
2 df_clean = df.dropna(axis=0)
```

- **info() & describe()**: Quick data profiling tools—`info()` shows data types and non-null counts, while `describe()` provides statistical summaries:

- **rename()**: Rename columns for clarity (e.g., rename 'Revenue USD' to 'Revenue'):

Code block

```
1 df = df.rename(columns={'Revenue USD': 'Revenue'})
```

- **sort_values()**: Sort the DataFrame by one or more columns (e.g., sort 'Sales' in descending order):

Code block

```
1 # Sort DataFrame by Sales (descending)
2 df_sorted = df.sort_values(by='Sales', ascending=False)
```

- **value_counts()**: Count occurrences of unique values in a categorical column (e.g., count users per category):

Code block

```
1 # Count unique values in 'Category' column
2 category_counts = df['Category'].value_counts()
```